

JotFlow Project Workflow

Current build status, operating flow, and missing work

Prepared from the local repository and local database on June 27, 2026.

Project stack: React/Vite frontend, Express backend, PostgreSQL database, JotForm Enterprise APIs, optional IMAP sender-mail sync, optional SMTP forwarding.

Executive Summary

JotFlow is currently built as a workflow monitoring and action dashboard for JotForm approval workflows. The backend syncs forms, submissions, workflow tasks, JotForm email records, user directory data, account history, and system logs into PostgreSQL. The frontend gives authenticated users dashboards for pending work, completed work, workflow details, JotForm email history, and admin views.

The main workflow path is functional: JotForm data is pulled into PostgreSQL, workflow tasks are normalized, task links are harvested from JotForm emails, and users can open or complete workflow tasks from the app. The email archive and forwarding path is also present. A separate sender-mail IMAP sync backend is present, but it is currently disabled in local config and the sender-mail table is empty.

7

JotForm submissions in local DB

23

Archived JotForm email records

0

Sender mailbox records synced

8

Workflow task email-log rows

5

JotForm directory users

4

Allowlist users from email recipients

High-Level Workflow

```
JotForm forms and workflows
|
v
Backend poller or admin sync
|
+-- Sync forms and submissions into jf_forms and jf_submissions
+-- Flatten workflow tasks into workflow_tasks JSONB
+-- Upsert task assignment records into email_logs
+-- Sync system, enterprise, and account history logs
+-- Archive JotForm-sent emails into jf_email_archive
+-- Extract action links from email bodies
+-- Save email recipients into allowlist_user
+-- Optionally forward archived emails via SMTP
+-- Optionally sync sender mailbox Sent folder through IMAP
|
v
React dashboard APIs
|
+-- User dashboard, pending queue, completed queue
+-- Director/admin dashboard and workflow details
```

```
    +-- My Emails and All Emails views
    +-- All Users directory view
    |
    v
User opens JotForm task link or completes task through backend
    |
    v
Backend calls JotForm API, updates PostgreSQL, emits realtime update
```

What Is Built

Area	Status	What exists now
Authentication and sessions	Built	Local login/signup, password reset, Microsoft OAuth route, session storage, role guards, CSRF origin checks, rate limiting, and secure headers.
PostgreSQL schema	Built	Core tables for users, profiles, organizations, submissions, approvals, forms, email logs, email archive, JotForm users, history logs, task tokens, sender-mail sync, and allowlist users.
JotForm profile support	Built	Backend profile registry supports multiple JotForm API profiles, with profile IDs stored across JotForm-sourced tables.
Submission sync	Built	Poller and admin sync read JotForm forms/submissions, map submission fields, persist workflow tasks, and update current pending approver/status fields.
Workflow task handling	Built	Workflow tasks are flattened for UI display, stored in <code>jf_submissions.workflow_tasks</code> , and task assignments are upserted into <code>email_logs</code> .
Task action links	Built	The system resolves JotForm task links from stored DB values, JotForm API fallback, prefill URLs, and JotForm email bodies.
Workflow actions	Built	Backend endpoints complete/approve/reject workflow tasks through JotForm, then update local submission state and approval history.
Dashboards	Built	Modern dashboard, director dashboard, workflow tracker, pending-with view, completed view, analytics, kanban, task detail sidebar, and workflow detail modal are present.
Email archive	Built	JotForm enterprise email events are pulled from system logs and <code>emailq/{id}</code> , stored in <code>jf_email_archive</code> , and shown in My Emails / All Emails views.
Email forwarding	Built	Archived emails can be copied to a configured mailbox through SMTP and tracked in <code>jf_email_forward_log</code> . Local config shows forwarding enabled.
Allowlist users	Built	The new <code>allowlist_user</code> table records usernames/mail IDs seen in JotForm-sent emails. Current local DB has 4 rows.
JotForm user directory	Built	Admin route and sync job populate <code>jf_users</code> ; frontend All Users view can search and display JotForm users.
Sender mailbox sync	Partially active	Backend table, IMAP sync library, CLI script, admin API, and poller toggle exist for syncing a sender mailbox Sent folder into <code>jotform_mail_sender</code> . Local config has it disabled and unpopulated.
Admin utilities	Built	Admin routes exist for users, emails, forms, sync, cleanup, profile sync, support messages, signature proxy, uploads, and config.

Current Local State

Item	Observed value	Meaning
<code>ENABLE_POLLER</code>	true	Backend is configured to run the periodic JotForm sync when the server starts.
<code>MAIL_SENDER_ENABLED</code>	false	Sender mailbox Sent-folder sync will not run automatically right now.
Sender mailbox credential	not configured	The sender mailbox account is configured, but IMAP password/access token is missing locally.
<code>EMAIL_FORWARD_ENABLED</code>	true	Backend-side forwarding of archived JotForm emails is enabled in local config.
<code>jotform_mail_sender</code>	0 rows	No actual sender mailbox email has been synced into the database yet.
<code>jf_email_archive</code>	23 rows	JotForm email archive has data from JotForm system logs/emailq.

Important distinction: JotForm email archive sync and sender mailbox Sent-folder sync are different.

`jf_email_archive` stores emails from JotForm's enterprise email log. `jotform_mail_sender` stores messages read directly from a mailbox through IMAP. The code for the mailbox path exists, but it is not enabled or populated in the current local setup.

What Is Missing or Incomplete

Gap	Priority	Required work
Sender mailbox sync not active	High	Set <code>MAIL_SENDER_ENABLED=1</code> , configure IMAP app password or OAuth access token, run <code>npm run sync:mail-sender</code> , and confirm rows appear in <code>jotform_mail_sender</code> .
Sender mailbox frontend view	Medium	Backend admin API exists, but no clear frontend page is wired for viewing <code>/api/admin/mail-sender</code> . Add an admin UI or connect it to the All Emails section.
Versioned migrations	Medium	The project applies a large idempotent <code>schema.sql</code> . Production would be safer with ordered migration files and a migration history table.
Automated test coverage	Medium	Some tests exist, but major flows need coverage: poller mapping, JotForm action calls, email archive ingestion, allowlist upsert, IMAP sender sync, and role-gated routes.
Operational monitoring	Medium	Add dashboard/alerts for poller health, last successful sync per profile, JotForm API failures, IMAP failures, SMTP forwarding failures, and stale email archive windows.
Retry/queue layer for background jobs	Medium	Background tasks currently run inside the Node process. A queue with job history would make retries and failure visibility stronger.
Webhook production hardening	Medium	Webhook registration code exists, but production depends on correct <code>PUBLIC_BASE_URL</code> , JotForm secret setup, and validation in the deployed environment.
Secrets and environment runbook	Low	Document required values for JotForm API keys, Microsoft OAuth, SMTP, IMAP, poller, and deployment. Include how to rotate keys safely.
Data cleanup and retention rules	Low	Define retention for archived email bodies, forwarded copies, mailbox messages, raw API payloads, support messages, and activity logs.

Recommended Next Steps

1. **Finish sender mailbox sync operationally.** Configure IMAP credentials, enable `MAIL_SENDER_ENABLED` , run the CLI sync, and verify `jotform_mail_sender` receives messages.
2. **Add or wire the sender-mail admin UI.** Reuse the existing `/api/admin/mail-sender` API so admins can inspect actual Sent-folder messages from the browser.
3. **Add tests for the new allowlist flow.** Cover recipient normalization, duplicate upsert behavior, and enrichment from `jf_users` .
4. **Add background sync health reporting.** Store last success/failure timestamps and show them in Settings/Admin.
5. **Move from one large schema file to migration files.** Keep `schema.sql` as a baseline, but use versioned migrations for future deploys.

Short Answer on Sender Mail

The project does have backend code to sync the sender mailbox into the database through IMAP. However, in the current local setup it is not running because `MAIL_SENDER_ENABLED` is false and no IMAP password/access token is configured. The local `jotform_mail_sender` table currently has 0 rows.